Best Practices for Using Claude Code in Construction Project Management

Catalyst / OBI — Construction Project Management Platform

Vue 3 | C# .NET 8 | SQL Server | v1.0 | March 2026

What this document is

This document defines how we use AI-assisted development at Catalyst / OBI. It ensures consistent, high-quality, and secure use of Claude Code across the team.

The single most important constraint

Claude's context window fills up fast — and performance degrades as it fills. Research shows 70% of tokens consumed in a typical Claude Code session are waste — Claude reading files it does not need. Every feature in Claude Code is essentially a context management tool. Keep this in mind at all times.

Quick start — Claude Code at OBI

5 rules

- Scope prompts to specific files
- Start a new session per task
- Use Plan mode before implementation
- Always verify outputs (tests or explicit checks)
- Use /clear after two failed attempts

5 commands

- /clear — reset context
- /compact — reduce noise
- /rewind — undo changes
- /hooks — inspect enforcement

- /rename — keep sessions organised

5 mistakes to avoid

- Vague prompts
- Kitchen sink sessions
- Repeated corrections in same session
- Bloated CLAUDE.md
- Trusting output without verification

1. **The layer model — what each part is for**

Claude Code is not one tool. It is a small operating system for AI-assisted development. Each layer solves a different problem. Using the wrong layer for a job is the most common mistake.

Layer	Purpose / where it lives
CLAUDE.md	Persistent instructions loaded at every session start. Defines stack, conventions, and what not to do. Keep under 200 lines — bloated files cause Claude to ignore rules. Lives at ./CLAUDE.md
Rules	Path-scoped instructions for specific areas of the codebase. Loaded only when relevant, reducing context overhead. Lives in .claude/rules/
Skills	Reusable playbooks for repeated workflows. A skill says "when this task appears, follow this exact sequence." Invoke with /skill-name. Lives in .claude/skills/
Hooks	Deterministic automation — guaranteed to run every time, regardless of what Claude is asked. Lives in .claude/settings.json

Subagents	Isolated Claude instances with their own context window, tools, and permissions. Keep verbose tasks out of your main session. Lives in .claude/agents/
MCP servers	Connect Claude to external tools — GitHub, Azure DevOps, databases, Jira. Use with restraint — every loaded MCP server adds token overhead on every prompt. Lives in .mcp.json
Plugins	Bundles of skills + hooks + agents + MCP, shareable across repos. Official Anthropic plugins available on GitHub. Lives in .claude/plugins/

2. **CLAUDE.md — the most important file**

Run /init inside Claude Code to auto-generate a starter file from your project structure. Refine it and commit to git so the whole team benefits.

What to include

Tech stack with reasons — not just what, but why, so Claude does not reverse your decisions:

```
## Tech stack

Frontend: Vue 3 (Composition API, <script setup>) — Options API not used

Backend: C# .NET 8 Web API, clean architecture (Controllers > Services > Repos)

Database: SQL Server — EF Core for ORM, Dapper for complex reporting queries

Auth: Azure AD via MSAL — do not implement custom auth flows

Storage: Azure Blob Storage for document uploads (drawings, specs, site photos)
```

Domain context — Claude needs to understand construction management to make sensible decisions:

```
## Domain context

Construction project management system. Core concepts:
```

```

Projects: top-level entity (a construction project)

Work packages: subdivisions of a project (civil, electrical, mechanical)

RFIs: formal contractor queries requiring an engineer response

Submittals: contractor documents for approval (shop drawings, specs)

Punch lists: defect lists at project handover

Daily reports: site supervisor logs (weather, labour, equipment, progress)

Cost codes: budget line items tracking expenditure against approved budgets
```

Coding conventions:

```
## Conventions

C#: PascalCase for classes/methods, _camelCase for private fields

All API responses use the ApiResponse<T> wrapper in Core/Models/ApiResponse.cs

Validation uses FluentValidation — never inline in controllers

Vue components use <script setup> — not the Options API

All DB changes through EF Core migrations — never modify schema directly

Async all the way down — no .Result or .Wait() on async calls
```

Do not section — often more valuable than positive instructions:

```
## Do not

Bypass the repository pattern — controllers must not access DbContext directly

Use synchronous DB calls — everything must be async/await

Add NuGet or npm packages without flagging it first

Write raw ADO.NET except in designated Dapper reporting repositories

Hardcode connection strings, Azure keys, or any credentials

Modify EF Core migrations once applied to any environment
```

Current focus — Claude has no memory between sessions. Two sentences here saves the first few exchanges every time:

```
## Current focus

[Update at session start — e.g.:

"Working on RFI overdue notifications in backend/notifications/.

Core RFI CRUD is stable. Submittal workflow is in progress, do not modify."]
```

@imports — keeps the root file tight while making deeper context available on demand:

```
## Key reference docs

API design standards: @docs/api-conventions.md

Database schema: @docs/schema-overview.md

Azure infrastructure: @docs/azure-setup.md

```

-

The pruning test

For every line in your CLAUDE.md, ask: "Would removing this cause Claude to make a mistake?" If not, cut it. Target under 200 lines. Move specific conventions to path-scoped rules files.

3. **Rules — scoping instructions to parts of the codebase**

Create .md files in .claude/rules/ and Claude loads them only when working in the relevant area — reducing context overhead on every other task.

```
.claude/rules/

  api-controllers.md       # loaded when Claude works in src/API/Controllers/

  vue-components.md        # loaded when Claude works in src/ClientApp/src/

  database-migrations.md   # loaded whenever Claude touches migrations or SQL

  reporting-queries.md     # loaded in Dapper reporting repositories
```

4. **Context management — the practice that changes everything**

Scope your prompts to specific files

A vague prompt triggers 20+ file reads. A specific prompt triggers 3-5. This is the biggest single improvement most teams can make.

Instead of	Use	Why it matters
Fix the RFI submission bug	Fix the bug in src/API/Controllers/RFIController.cs — POST /api/rfis returns 500 when the project has no active work packages	3-5 file reads instead of 20+
Add validation to the submittal form	Add validation to src/ClientApp/src/components/submittals/SubmittalForm.vue — required fields are title, contractor, and due date	Claude goes directly to the right file
Why is the daily report query slow?	The query in src/API/Infrastructure/Reporting/DailyReportRepository.cs is slow on large projects. Investigate and optimise.	Targets the exact file, no exploration needed

Token discipline — what actually drives cost and quality

- File reads are the primary token driver
- Vague prompts trigger large-scale file exploration
- MCP servers increase baseline token usage on every prompt

Rule of thumb: If Claude reads more than 10 files for a task, the prompt is too vague.

Key context commands

- **/clear** Reset the context window between unrelated tasks. CLAUDE.md reloads automatically — you keep project context, lose conversation history only.
- **/compact** Compress context without losing the thread. Use: /compact focus on the RFI notification changes we discussed

- **/btw** Ask a quick question that never enters conversation history. Use: /btw what is the EF Core syntax for a soft-delete global query filter?
- **/clear (after failed corrections)** After two failed corrections on the same issue, run /clear and restart with a better prompt. A clean session beats a polluted one.

Name sessions early so they are findable later:

```
claude -n rfi-overdue-notifications   # name at startup
```

```
/rename rfi-overdue-notifications      # rename during a session
```

5. **The recommended workflow — explore, plan, code, commit**

Separating research from implementation produces significantly better results. Use Shift+Tab to cycle through permission modes in the terminal.

Step	Mode	What to do
1. **Explore**	Plan Mode (Shift+Tab twice — shows ⏸ plan mode on)	Ask Claude to read relevant files and understand the current state. No changes are made. Example: Read src/API/Controllers/RFIController.cs and src/API/Services/RFIService.cs. What would need to change to add overdue notifications?
2. **Plan**	Plan Mode	When requesting a plan, always require at least two implementation approaches. For each approach, Claude must: • Describe the approach • Analyse trade-offs (complexity, performance, risk)

		• Identify risks and unknowns Claude must recommend a preferred approach with justification before any implementation begins. Press Ctrl+G to open the plan in your editor and refine it. When you accept the plan the session is automatically named from its content.
3. **Implement**	Normal Mode (Shift+Tab back)	Implement from the plan. Always include a verification step — a test to run or a condition to check. Claude performs significantly better when it can verify its own work.
4. **Commit**	Normal Mode	Ask Claude to commit with a descriptive message and open a PR. Example: Commit with a descriptive message and open a PR.

Rewinding when things go wrong: every file edit creates a checkpoint. Press Esc+Esc or run /rewind to restore conversation and/or code to any previous state. Checkpoints persist across terminal closures.

6. **Skills — packaging repeated workflows**

Skills are SKILL.md files in .claude/skills/. They are the right tool for workflows that should follow the same sequence every time. A rule says "here is how we do things". A skill says "when this task appears, follow this exact playbook".

Example: EF Core migration skill

```
# .claude/skills/new-migration/SKILL.md

---

name: new-migration

description: Create a new EF Core migration for a SQL Server schema change.

  Use when adding tables, columns, indexes, or constraints.

disable-model-invocation: true

---

Create a new EF Core migration for: $ARGUMENTS



1 - Update the relevant entity class(es) in src/API/Core/Entities/

2 - Update ApplicationDbContext.cs if needed

3 - Run: dotnet ef migrations add <DescriptiveName> --project src/API

4 - Review the generated migration file — confirm it matches the intended change

5 - Update docs/schema-overview.md with the change

6 - Do not apply the migration — confirm with the developer first


```

Invoke with: /new-migration add ContractorContact table with FK to Projects

Example: clean architecture endpoint scaffold skill

```
# .claude/skills/new-endpoint/SKILL.md

---

name: new-endpoint

description: Scaffold a new API endpoint following clean architecture conventions.

disable-model-invocation: true

---

Scaffold a new API endpoint for: $ARGUMENTS



1. Entity in src/API/Core/Entities/

```

```
2. Request/response DTOs in src/API/Core/DTOs/

3. Interface in src/API/Core/Interfaces/

4. Service implementation in src/API/Application/Services/

5. Repository in src/API/Infrastructure/Repositories/

6. Register in src/API/Program.cs

7. Controller action in src/API/Controllers/

8. FluentValidation validator in src/API/Application/Validators/

9. Unit tests in tests/UnitTests/

10. Confirm dotnet test passes before finishing
```

Invoke with: /new-endpoint GET /api/projects/{id}/punchlist-summary

disable-model-invocation: true

Add this to any skill with side effects — migrations, deployments, commits — so Claude cannot trigger it automatically without you explicitly invoking it.

7. **Hooks — the enforcement layer**

Hooks are deterministic. CLAUDE.md is advisory.

CLAUDE.md instructions can be overlooked as context fills. Hooks always fire — regardless of what Claude is asked to do. Use hooks for anything that must happen every time without exception.

Hooks live in .claude/settings.json (project-level, commit to git) or ~/.claude/settings.json (personal, all projects).

Hooks enforce correctness automatically. Build, type-check, and tests should run without manual steps.

Hook events

Event	When it fires	Typical use

PreToolUse	Before a tool call	Block dangerous commands, validate before writing
PostToolUse	After a tool call succeeds	Auto-build check, lint, type-check
SessionStart	Session begins	Load environment, inject context
Notification	Claude needs attention	Desktop alerts during long tasks
SubagentStart/Stop	Subagent lifecycle	Setup/teardown resources

Practical examples

Run dotnet build check after every C# file write:

```
{
  "hooks": {
    "PostToolUse": [{
      "if": "Edit(src/API/**/*.cs)|Write(src/API/**/*.cs)",
      "hooks": [{ "type": "command",
        "command": "cd src/API && dotnet build --no-restore -v q" }]
    }]
  }
}
```

Run Vue type-check after every component edit:

```
{
  "hooks": {
    "PostToolUse": [{
      "if": "Edit(src/ClientApp/**/*.vue)|Edit(src/ClientApp/**/*.ts)",
      "hooks": [{ "type": "command",
        "command": "cd src/ClientApp && npm run type-check" }]
    }]
  }
}
```

Warn before Claude touches EF Core migrations:

```
{
  "hooks": {
    "PreToolUse": [{
      "if": "Edit(src/API/Infrastructure/Migrations/*)",
```

```
      "hooks": [{ "type": "command", "command":
        "echo 'WARNING: Only edit migrations not yet applied to any environment' && exit 1" }]
    }]
  }
}
```

Desktop notification when Claude needs your attention (macOS):

```
{
  "hooks": {
    "Notification": [{
      "hooks": [{ "type": "command", "command":
        "osascript -e 'display notification \"Claude needs attention\" with title \"Claude Code\"'" }]
    }]
  }
}
```

AI reviewing AI — use Haiku to review every file write for security issues:

```
{
  "hooks": {
    "PostToolUse": [{
      "if": "Edit|Write",
      "hooks": [{ "type": "prompt", "model": "claude-haiku-4",
        "prompt": "Review this change for security issues: $ARGUMENTS.
          Check for SQL injection in Dapper queries, missing [Authorize] attributes,
          and hardcoded credentials. Return BLOCK with explanation or ALLOW." }]
    }]
  }
}
```

Exit codes: 2 = blocks the action, 1 = warning only (proceeds), 0 = allows. Run /hooks to see all active hooks.

8. **Subagents — protecting your context window**

Subagents run in their own isolated context window. Anything verbose — codebase research, test runs, security audits — should be delegated so it does not pollute your main session.

Subagents are not just for context isolation

They are also a tool for separating thinking:

- Perspective separation — implement vs review
- Domain separation — frontend, backend, database

- Goal separation — exploration, optimisation, validation

Using subagents this way produces higher quality results than a single session handling everything.

Team code reviewer subagent

Create .claude/agents/code-reviewer.md and commit to git — everyone gets the same review behaviour:

```
---

name: code-reviewer

description: Expert code reviewer for the Catalyst/OBI platform. Proactively

  reviews C# .NET and Vue code for quality, security, and project conventions.

  Use after writing or modifying code.

tools: Read, Grep, Glob, Bash

model: inherit

memory: project

---

You are a senior developer reviewing code for a construction project management

system built with C# .NET 8, Vue 3, and SQL Server.



When invoked: run git diff, focus on modified files, begin review immediately.



Check: clean architecture respected (no DbContext in controllers), all async

methods awaited, FluentValidation used, Dapper queries parameterised, API

responses use ApiResponse<T>, Vue uses <script setup>, no hardcoded credentials,

[Authorize] attributes present on new controller actions.



Feedback in order: Critical (must fix) > Warnings (should fix) > Suggestions.
```

```

Include specific code examples showing how to fix each issue.
```

Invoking subagents

```
# Natural language

Use the code-reviewer subagent to check the RFI notification service



# @-mention — guarantees this subagent runs

@code-reviewer review the changes I just made to the submittal workflow



# Parallel research across the stack

Research how project permissions work — use separate subagents to investigate

the C# permission service, the database schema, and the Vue route guards.

Report back before we make any changes.
```

Writer/reviewer pattern

One session implements, another reviews with fresh eyes and no bias toward code it just wrote:

```
# Session A: implement

claude -n implement-punch-list-export



# Session B: review (no bias toward code it just wrote)

claude -n review-punch-list-export

# Ask: Review the punch list export in @src/API/Application/Services/PunchListService.cs

#       Focus on error handling, large dataset performance, and authorisation checks.
```

9. **Session management**

Naming and resuming

```
claude -n rfi-overdue-notifications      # name at startup

/rename rfi-overdue-notifications         # rename during a session
```

```
4
5
6
7 claude --continue                          # resume most recent
  session
8
9 claude --resume rfi-overdue-notifications # resume by name
10
11 claude --resume                           # open interactive session
   picker
12
13 claude --from-pr 83                       # resume a session linked to
   a PR
14
15 claude --continue --fork-session          # try a different approach
   without
16
17                                           # losing where you are
```

Parallel work with worktrees

Run two features in parallel without them colliding — each gets its own working directory and branch:

```
claude --worktree feature-rfi-notifications   # isolated branch
```

```
claude --worktree bugfix-submittal-upload      # second parallel
session
```

Add .claude/worktrees/ to your .gitignore.

10. Security habits

Review every diff before merging

Claude Code can touch controllers, services, repositories, migrations, and Vue components all in one pass. Treat its output like a PR from a junior developer — it goes through review before merge.

- **--permission-mode plan** Use in unfamiliar areas or near authentication/billing. Claude can read and analyse but cannot make changes without your approval.
- **[Authorize] on every new controller action** The most common mistake when Claude scaffolds an endpoint. Make it part of your review checklist or enforce it with a prompt-type hook.
- **Audit MCP servers before connecting** Untrusted servers can introduce prompt injection — malicious instructions embedded in data Claude reads through the server.

- **security-guidance plugin** Install the official Anthropic plugin for zero-config security pattern detection on every file write. Blocks eval(), .innerHTML=, dangerouslySetInnerHTML, child_process.exec, and more automatically.

Install the security-guidance plugin per-project (commit to git — team gets it automatically):

```
git clone --filter=blob:none --sparse \

  https://github.com/anthropics/claude-code.git /tmp/cc-plugins

cd /tmp/cc-plugins && git sparse-checkout set plugins/security-guidance



mkdir -p .claude/plugins

cp -r plugins/security-guidance .claude/plugins/

git add .claude/plugins/security-guidance

git commit -m "Add security-guidance Claude Code plugin"
```

Verify with /hooks — you should see security_reminder_hook.py listed under PreToolUse.

11. When Not to use Claude Code

Claude Code is powerful but not universal. Avoid using it in the following scenarios without tight control:

- Large cross-cutting refactors without precise scoping
- Security-critical logic (auth, permissions, billing) without human-first design
- Performance-critical code without profiling and benchmarking
- Financial or compliance-sensitive calculations where correctness is paramount

In these cases, use Claude for exploration and planning only, not direct implementation.

12. Secrets and .env files — protecting sensitive configuration

Claude Code does not exclude .env files by default

Claude has read access to your entire project directory from the moment you run claude. Without explicit protection, it will read appsettings.Development.json, .env files, and any file

containing credentials — and those values will sit in the context window for the duration of the session.

The .gitignore misconception

Many developers assume that because appsettings.Development.json or .env is in .gitignore, Claude Code will ignore it too. It will not. .gitignore tells git what not to track. Claude Code ignores .gitignore entirely for file access. You need explicit deny rules.

Step 1 — Add deny rules to .claude/settings.json

This is the primary protection layer. Commit this file to git so the whole team gets the rules automatically:

```
// .claude/settings.json

{

  "permissions": {

    "deny": [

      "Read(./appsettings.Development.json)",

      "Read(./appsettings.Production.json)",

      "Read(./appsettings.Staging.json)",

      "Read(./**/appsettings.*.json)",

      "Read(./.env)",

      "Read(./.env.*)",

      "Read(./src/ClientApp/.env)",

      "Read(./src/ClientApp/.env.*)",

      "Read(./secrets/**)",

      "Read(./**/*.pfx)",

      "Read(./**/*.key)"

    ]

  }

}
```

Step 2 — Add .claude/settings.local.json to .gitignore

This file is for personal overrides that should never be committed. Add it to .gitignore immediately:

```
# .gitignore

.claude/settings.local.json

.claude/worktrees/
```

If a developer occasionally needs to temporarily relax a deny rule for a specific debugging session, they do it in settings.local.json — the decision stays on their machine and never enters git history.

The Bash subprocess gap — a critical nuance

Deny rules only block Claude's built-in Read tool

The official Anthropic docs are explicit: a Read(./.env) deny rule blocks the Read tool but does NOT prevent Claude running 'cat .env' in Bash. For full protection you need a PreToolUse hook or the sandbox.

Add a PreToolUse hook to intercept Bash commands that touch sensitive files:

```
// .claude/settings.json — add alongside your deny rules

{

  "hooks": {

    "PreToolUse": [{

      "if": "Bash",

      "hooks": [{ "type": "command", "command":

        "echo '$CLAUDE_TOOL_INPUT' | grep -iE


'(appsettings\.Development|appsettings\.Production|\.env|secrets/)

         ' && echo 'Blocked: sensitive file access' && exit 2 || exit
0" }]

    }]

  }

}
```

Telling Claude what it needs without showing secrets

When Claude needs configuration context to do its job — for example, to write code that reads a connection string — describe the structure without the values:

```
// Good — Claude gets the context it needs without seeing the secret

The database connection string is in appsettings.Development.json under

ConnectionStrings:DefaultConnection. I am not sharing the actual value.

Write the code to read it using IConfiguration in the standard .NET way.



// Also good — Claude writes correct code without needing the actual key

The Azure Blob Storage connection string is in appsettings.Development.json

under AzureStorage:ConnectionString. Write a service that reads this value

at startup and injects it into BlobServiceClient.
```

This is the right pattern throughout: Claude writes code that reads from configuration correctly, without ever needing to see the actual secret values.

What the complete settings.json looks like

Combining deny rules, the Bash hook, and other project hooks in one file:

```
// .claude/settings.json — commit this to git

{

  "permissions": {

    "deny": [

      "Read(./appsettings.Development.json)",

      "Read(./appsettings.Production.json)",

      "Read(./appsettings.Staging.json)",

      "Read(./**/appsettings.*.json)",

      "Read(./.env)",

      "Read(./.env.*)",

      "Read(./src/ClientApp/.env*)",

      "Read(./secrets/**)"

    ]

  },

  "hooks": {

```

```
    "PreToolUse": [

      {

        "if": "Bash",

        "hooks": [{ "type": "command", "command":

          "echo '$CLAUDE_TOOL_INPUT' | grep -iE 'appsettings\.Dev|appsettings\.Prod|\.env' && exit 2 || exit 0" }]

      },

      {

        "if": "Edit(src/API/Infrastructure/Migrations/*)",

        "hooks": [{ "type": "command",

          "command": "echo 'WARNING: Only edit unapplied migrations' && exit 1" }]

      }

    ],

    "PostToolUse": [

      {

        "if": "Edit(src/API/**/*.cs)|Write(src/API/**/*.cs)",

        "hooks": [{ "type": "command",

          "command": "cd src/API && dotnet build --no-restore -v q" }]

      }

    ]

  }

}
```

Protection levels — choose what suits your context

Protection	What it covers	Effort
Deny rules in settings.json (commit to git)	Blocks Claude's built-in Read tool from accessing sensitive files	Low — add once, whole team gets it
settings.local.json in .gitignore	Prevents personal overrides leaking into git history	Low — one line in .gitignore

PreToolUse hook on Bash	Blocks shell subprocess access (cat, type, Get-Content)	Medium — add to settings.json
Enable /sandbox	OS-level isolation — blocks all processes from accessing specified paths	Medium — requires configuration
Devcontainer	Maximum isolation — firewall, network controls, fully sandboxed environment	High — team tooling change

13. Common failure patterns

The kitchen sink session

What happens: you fix a C# bug, add a Vue component, ask a database question, all in one session. Context fills with irrelevant information and Claude starts making mistakes.

Fix: run /clear and start a new named session for each distinct piece of work.

Correcting over and over

What happens: Claude scaffolds a controller action incorrectly — accessing DbContext directly. You correct it. Still wrong. Third correction. The context is now full of failed attempts and Claude is confused.

Fix: after two failed corrections run /clear. Restart with a better prompt that references the existing correct pattern explicitly — for example, "follow the exact same pattern as GetById in RFIController.cs".

Claude performs significantly better in a fresh session than in a polluted one.

Bloated CLAUDE.md

What happens: the team keeps adding conventions and edge cases. The file grows to 500+ lines. Claude starts ignoring important rules because they are buried.

Fix: prune ruthlessly. Apply the pruning test. Move C#-specific conventions to .claude/rules/api-controllers.md and Vue conventions to .claude/rules/vue-components.md.

Vague prompts

What happens: "Fix the submittal upload bug" triggers Claude to read every file in the solution. Tokens explode. Usage limits hit.

Fix: always reference the specific file, describe the symptom, and state what correct looks like. "Fix the file upload in src/ClientApp/src/components/submittals/SubmittalUpload.vue — the upload succeeds but the file does not appear in the submittal detail view. The API endpoint is POST /api/submittals/{id}/attachments."

Trust without verify

What happens: Claude adds a new project role check and it looks correct. Six months later someone reports subcontractors can see all project financials. The [Authorize] attribute was on the wrong method.

Fix: always give Claude a verification criterion. "Write integration tests verifying that a Viewer role receives 403 on the POST, PUT, and DELETE cost code endpoints. Run dotnet test --filter Category=Integration and confirm they pass."

The harness gap

What happens: two developers on the team, same Claude model, wildly different quality results. One is shipping fast; the other is frustrated.

Fix: standardise the harness. A well-maintained CLAUDE.md, a shared code-reviewer subagent, task skills for repeated workflows, and the settings.json with deny rules and hooks — committed to git, available to everyone.

14. Prompt patterns that work

Specify the file, the symptom, and what correct looks like

```
The PUT /api/projects/{id}/status endpoint in src/API/Controllers/ProjectController.cs

returns 200 even when the status transition is invalid (e.g. Closed back to Active).

Add validation for valid transitions per docs/project-status-workflow.md.

Write tests for valid and invalid transitions. Run dotnet test and confirm they pass.
```

Point Claude to an existing pattern

```
Look at how the RFI module is structured across RFIController.cs, RFIService.cs,

and RFIRepository.cs. Follow exactly the same clean architecture pattern to
```

```

implement a Punch List module.
```

Let Claude interview you for a complex feature

```
I want to add a document approval workflow to the Catalyst submittals module.

Interview me in detail using the AskUserQuestion tool. Ask about the approval steps,

who can approve at each step, what happens on rejection, email notifications, and

how it integrates with the existing submittal status model.

Once covered, write a complete spec to docs/submittal-approval-spec.md.
```

Then start a fresh session to implement from the spec — clean context, focused on execution.

Further reading

- Official best practices: Best Practices for Claude Code - Claude Code Docs
- Common workflows: Common workflows - Claude Code Docs
- Skills reference: Extend Claude with skills - Claude Code Docs
- Hooks reference: Hooks reference - Claude Code Docs
- Subagents: Create custom subagents - Claude Code Docs
- Settings and deny rules: Claude Code settings - Claude Code Docs
- Security: Security - Claude Code Docs
- Official plugins: claude-code/plugins at main · anthropics/claude-code

This is a living document

Please improve it. Add patterns that work for the Catalyst/OBI codebase, remove things that don't, and update the examples with real conventions as they evolve. The value compounds over time.